Final Report: Development of Agent's Control System

Course Number: ENPH353
Date: December
Authors: Kamyar Momeni, Robert Liu
Github Link

Contents

1 Introduction

The purpose of this report is to provide a comprehensive overview of the development and implementation of an autonomous control system for a robotic agent designed for the *Fizz Detective* competition. The competition challenges participants to design and deploy a robot capable of navigating a simulated environment while adhering to traffic rules and detecting specific clues to maximize the overall score. This report describes the steps taken to ensure the system's functionality, the methods utilized to achieve reliability and accuracy, and the results obtained during testing. It also identifies areas for future improvement and reflects on the challenges encountered during development.

2 Background

The *Fizz Detective* competition was a test of robotic driving and computer vision abilities within a simulated Gazebo environment. The robot must complete its tasks within a strict 4-minute time limit, requiring teams to develop efficient algorithms for navigation and clue detection. Points are awarded for identifying and sending up to eight clues, with scoring weighted to favor early clues (+6 points each for the first six) and additional points for the last two (+8 points each). The competition imposes penalties for rule violations, including -5 points for traffic infractions or collisions with pedestrians, vehicles, or Baby Yoda, and -2 points for driving off-road or respawning outside designated areas. Successfully navigating the tunnel section without respawns provides a +5 point bonus. The competition map is divided into distinct areas, each presenting unique challenges. For instance, the starting area offers a straightforward environment for line-following calibration, while the urban area introduces dynamic obstacles and strict adherence to traffic rules. The clue area requires precise detection algorithms to identify color patterns associated with clues. Additionally, the tunnel section demands exceptional alignment and control, and the final section includes complex obstacles to test the system's robustness under edge conditions.

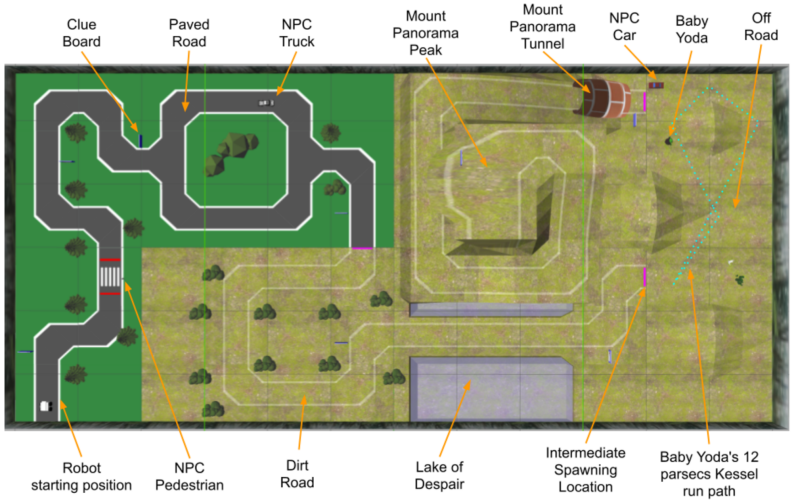

Figure 1: Competition map labeling different areas and their unique challenges.

2.1 Project Objectives

The project aims to design and implement a control system capable of achieving the competition's objectives. These include robust navigation, accurate clue detection, compliance with traffic rules, and modular software architecture. A key focus is the use of computer vision and neural networks for clue recognition while optimizing performance in challenging scenarios such as narrow roads and dynamic obstacles.

3 Contribution Split

Kamyar Momeni was responsible for robot driving, which included designing algorithms for board detection and line following. Robert Liu contributed by integrating it with ROS topics, and clue detection and CNN training for clue detection. He also developed the graphical user interface (GUI) and debugging tools, ensuring smooth operation and rapid error identification. Both members played integral roles in the project, leveraging their expertise to create a cohesive and effective system.

4 Software Architecture

The system architecture is based on a modular design that integrates perception, decision-making, and control. The perception module processes camera inputs to extract meaningful information, such as lane markings and clues.

The system is divided into parts to enhance clarity and maintainability, with perception, planning, and control operating independently but in coordination. A custom PyQt-based GUI was developed to provide real-time visual feedback and debugging capabilities, including camera feeds, intermediate masks, and system status messages. This modular approach ensures scalability and adaptability for future enhancements.

5 Driving Module

The driving module is the primary component of the robot's navigation system. Implemented as a ROS node, it processes real-time camera input and dynamically controls the robot's velocity to navigate through various operational zones. By integrating computer vision techniques and adaptive steering mechanisms, the module ensures smooth navigation through the simulation environment.

The module subscribes to the `/B1/camera1/image_raw` topic to receive image data and publishes velocity commands to the `/B1/cmd_vel` topic. Velocity adjustments are achieved using ROS `Twist` messages, where linear and angular velocities are updated based on detected navigation cues.

5.1 Line Following

Line following is a key navigation task where the robot detects and tracks a designated line to stay on course. Using HSV color masking, the robot identifies the line and calculates how far it is from the center of the camera frame. To correct its path, the robot adjusts its speed and steering based on this offset, smoothly aligning itself with the line.

The process starts by converting the camera image into the HSV color space, which makes it easier to isolate the line using a specific color range. To reduce noise and highlight the line, the system applies smoothing techniques like erosion and dilation. The following function showcases the logic for line following:

Listing 1: Line Following Implementation

```
def detect_navigation_line(self, frame, lower_color, upper_color):
    hsv_frame = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
    mask = cv.inRange(hsv_frame, lower_color, upper_color)

    kernel = np.ones((5, 5), np.uint8)
    mask = cv.erode(mask, kernel, iterations=1)
    mask = cv.dilate(mask, kernel, iterations=2)

    frame_height, frame_width = mask.shape
    scanline_y = int(0.75 * frame_height)
    line_indices = np.where(mask[scanline_y, :] > 0)[0]

    if len(line_indices) > 0:
        line_position = int(np.mean(line_indices))
        alignment_error = (line_position - frame_width // 2) / (
            frame_width // 2)
        forward_speed = 2.5 * (1 - abs(alignment_error))
        angular_speed = -20 * alignment_error
        self.publish_velocity(forward_speed, angular_speed)
```

The alignment error represents how far the detected line is from the center of the camera frame. To stay on track, the robot gently adjusts its angular velocity based on this error, turning just enough to correct its path while continuing to move forward smoothly.

5.2 Dynamic Zone Transitions

To progress through different operational zones, the robot detects transition markers in the form of pink lines. The detection logic has a confidence-based system to avoid false detections. It also monitors the bottom part of the frame and adjusts HSV threshold values based on the average color it sees.

The implementation identifies pink areas exceeding a defined pixel threshold. When the pink pixels exceed the threshold for multiple frames, it finally passes the confidence level and allows for the zone transition.

Listing 2: Zone Transition Detection

```
def detect_transition_line(self, frame):
    hsv_frame = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
    roi = hsv_frame[int(0.75 * frame.shape[0]):, :]

    mean_hue = int(np.mean(roi[:, :, 0]))
    lower_pink = np.array([max(mean_hue - 15, 140), 50, 50])
    upper_pink = np.array([min(mean_hue + 15, 179), 255, 255])

    mask = cv.inRange(hsv_frame, lower_pink, upper_pink)
```

```
    pink_area = np.sum(mask > 0)

    if pink_area > 2000:
        self.zone_confidence_score += 1
    else:
        self.zone_confidence_score = max(0, self.
            zone_confidence_score - 1)

    if self.zone_confidence_score > 5:
        print("Pink transition line detected!")
        self.zone_confidence_score = 0
        return True
```

5.3 Tunnel Alignment

Navigating through tunnels requires precise alignment to avoid collisions. The robot uses moment-based center detection to calculate the horizontal offset from the tunnel's center and adjusts its movement accordingly. A mask isolates the tunnel's features based on HSV color thresholds, and the region of interest (ROI) focuses on the lower portion of the frame.

The following function demonstrates tunnel alignment:

Listing 3: Tunnel Alignment Implementation

```
def align_with_tunnel(self, frame):
    hsv_frame = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
    lower_tunnel = np.array([5, 72, 156])
    upper_tunnel = np.array([14, 255, 210])
    mask = cv.inRange(hsv_frame, lower_tunnel, upper_tunnel)

    roi = mask[int(0.6 * frame.shape[0]):, :]
    moments = cv.moments(roi)

    if moments['m00'] > 0:
        center_x = int(moments['m10'] / moments['m00'])
        alignment_error = (center_x - frame.shape[1] // 2) / (frame.
            shape[1] // 2)
        forward_speed = 2.5 * (1 - abs(alignment_error))
        angular_speed = -15 * alignment_error
        self.publish_velocity(forward_speed, angular_speed)
```

The robot calculates the tunnel's center using the image moments and applies proportional control to adjust its alignment error, ensuring smooth navigation within confined spaces.

5.4 Hill Navigation

To drive over the hill, the robot uses edge detection to identify the path boundaries. By analyzing the slopes of detected lines, the robot determines its steering direction and adjusts its velocity to maintain stability.

The implementation uses the Canny edge detection method followed by Hough line detection to calculate average slopes:

Listing 4: Hill Navigation Using Edge Detection

```
def detect_hill_path(self, frame):
    roi = frame[int(frame.shape[0] / 2):, :]
    gray_frame = cv.cvtColor(roi, cv.COLOR_BGR2GRAY)
    edges = cv.Canny(gray_frame, 50, 150)

    lines = cv.HoughLinesP(edges, 1, np.pi / 180, 50, minLineLength
       =50, maxLineGap=10)
    if lines is not None:
        slopes = [(y2 - y1) / (x2 - x1 + 1e-6) for x1, y1, x2, y2 in
            lines[:, 0]]
        avg_slope = np.mean(slopes)
        if abs(avg_slope) < 0.5:
            self.publish_velocity(3.0, 0)
        elif avg_slope > 0.5:
            self.publish_velocity(3.0, -10)
        elif avg_slope < -0.5:
            self.publish_velocity(3.0, 10)
```

The hill detection code works by analyzing the lower half of the frame to spot edges using Canny edge detection. It then identifies straight lines with the Hough Line Transform and calculates their slopes. If the slopes are mostly flat, the robot keeps moving straight. However, if the lines show a tilt to the left or right, the robot adjusts its steering to stay on track. This way, the robot can smoothly handle hills or uneven surfaces without losing its direction.

5.5 Summary

The driving module combines computer vision, proportional control, and dynamic thresholding to enable the robot to autonomously navigate through complex zones. By leveraging line detection, tunnel alignment, and hill traversal logic, the module ensures robust and reliable performance in varying environments.

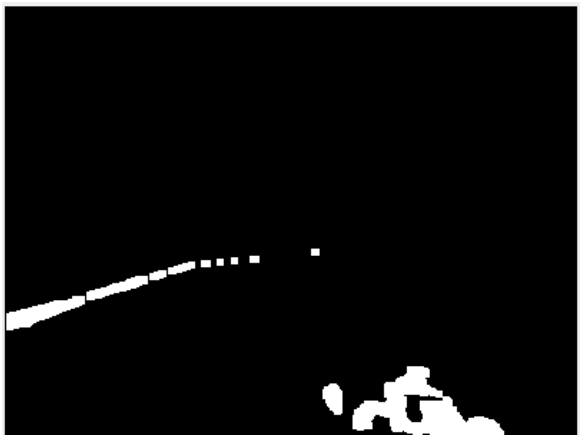

Figure 2: Binary threshold from line following.

Figure 3: Binary threshold of the pink respawn lines used for determining zones.

Figure 4: Binary threshold for the tunnel.

6 Clue Plate Recognition Module

To go about this task we followed Lab 6 quite to the tee, however there were a couple nuances that we had to take into account.

1. There was not a set size on the clue, we did not know if the clue would be 1 character long or 12 characters long, obviously we could assume that there was a clue and the clue was bounded inside the clue board, meaning that we could assume that each clue was 1-12 characters long.

2. There were also spaces, meaning that we had to be sure that if we did detect a space, to ignore it and not sent it in the message. We would need to develop a method that ignores these spaces and if we were to used fixed spacing, to ensure that it does not detect some character by accident.

To go about this we would first need to create our clue board to train off of, this would be simple as the generation of the clue boards was in the competition file, we were able

to snag what the files that we needed and generated random letters as the clues. Then applied some gaussian noise to recreate some noise that would be experienced by the camera in Gazebo.
Processing the images and recognizing characters was a more complicated process. We are going to talk about how we did it in competition however it is quite similar to how I trained it however, there was obviously the addition of having to find the clueboard in the image and introduce homography. It required multiple steps in processing the image and this is how it flowed in competition:

1. Find a quadrilateral that had a blue outline based on the HSV filtering. This did require some nuance as some signas were in the direction of the sun and had blue values that were far lighter than the others, thus we had to manually adjust those so the quadrilateral detection would work.

2. Once and if a blue quadrilateral had been found, it applies homography and dynamic cropping in order to attempt to get the original image of the sign. This simply gets rid of the blue outline of the homography image making it as similar to the training data as possible and sets an easy path to character detect.

3. Extracting the regions of interest, this would simply be the bottom half of the clueboard, however I also thought of adding the top right section just in case we wanted to change our strategy and use the top right to give us hints about where we are and what clue we are on.

4. Then we preprocess each clue area by simply adding a blur and binary inverse thresholding this also cropped the images edges just in case there was extra blue that might be recognized as a character. This was fine since all the characters were near the middle of the clue board.

5. Then we had to process the image and what was essentially done here was that we found countours in the binary inverse and considered those characters. Once we found all the contours, we would put them in a similar form to how the model was trained, being a 28x28 character image, and adding quite a lot of padding to follow what the model was trained on. Once all the character countours were processed, we sent all the information to the score tracker.

For preprocessing there was also this method that was important especially when the quality of the camera or image was low:

Listing 5: Morphing and Connecting broken letters

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
```

This was actually pivotal in some instances where some characters such as 'M' were often cut up into 2 or even three characters based on my contouring method allowing for some improvements in accuracy and allowed for more noise removal.
This was quite similar to how it was trained, training simply took the clue plate generation that was in the 2024 competition folder and created many clue boards that were filled with random letters. The regions of interest and preprocessing was very similar with some slight differences in that some HSVs in some cases would be off and cause issues. There were some cases where the model was struggling with Cs and Gs, as well as sometimes

getting 1 and I confused, so to clean some of those confusions, I had increases sample sizes of the more confusing letters to increase their distinguish ability.
This process was to happen at the end of the run since it was computationally expensive and often times cause the robot to lag, and sometimes fall of course. Not only that, it was easier to implement it at the end since then we would not have to run the command many times over and the implementation of the separately developed driving and clue detection was easier and faster to implement together and time was of essence in this situation. To do this we had to incorporate a camera capture method for the signs that would take pictures of the signs as we were going and save them for processing later. One big thing that we noticed was that when we increased the resolution of our camera our results were much better. This makes sense as there was less of those low quality issues such as M being read of 2 or 3 letters however if we used it all the time, we would experience too much lag, and the robot could not drive reliably. So taking inspiration from other teams, we had 2 cameras, one lower quality that was used for driving and one higher quality one for clue detection that allowed us to get the best of both worlds.

6.1 Data Acquisition and Preprocessing

The dataset consists of 28×28 grayscale character images with one-hot encoded labels. It was split into training (80%) and validation (20%) sets using `train_test_split`. Preprocessing included normalization to $[0, 1]$, reshaping to $(28, 28, 1)$, and one-hot encoding for labels.

6.2 Neural Network Architecture

The CNN has an input layer ($28 \times 28 \times 1$), two convolutional layers (32 and 64 filters, 3×3 kernel), max-pooling layers (2×2), a flatten layer, a dense layer with 128 neurons and ReLU, a dropout layer (rate 0.5), and a softmax output layer for classification. Below is the summary:

```
Layer (type)          Output Shape        Param #
conv2d                (None, 26, 26, 32) 320
max_pooling2d         (None, 13, 13, 32) 0
conv2d_1              (None, 11, 11, 64) 18496
max_pooling2d_1       (None, 5, 5, 64)   0
flatten               (None, 1600)       0
dense                 (None, 128)        204928
dropout               (None, 128)        0
dense_1               (None, num_classes)num_classes * 128 + num_classes
Total params: 223,744
```

6.3 Training Parameters

The model was trained with the Adam optimizer (default learning rate), categorical crossentropy loss, and accuracy metric, for 10 epochs and a batch size of 32. The dataset contained 80% training and 20% validation samples.

6.4 Results and Analysis

The figures below summarize the results. The loss and accuracy plots show training and validation performance over epochs, while the histogram illustrates the dataset composition. The confusion matrix highlights prediction performance.

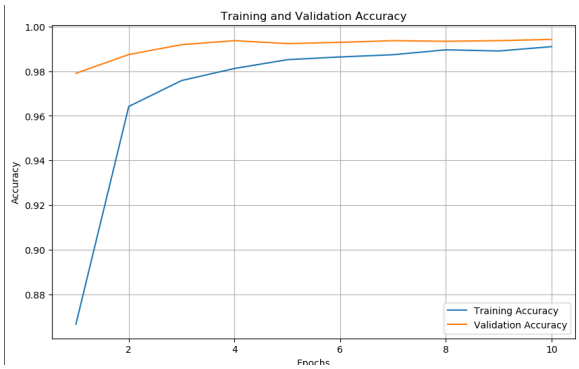

Figure 5: Accuracy vs Epochs

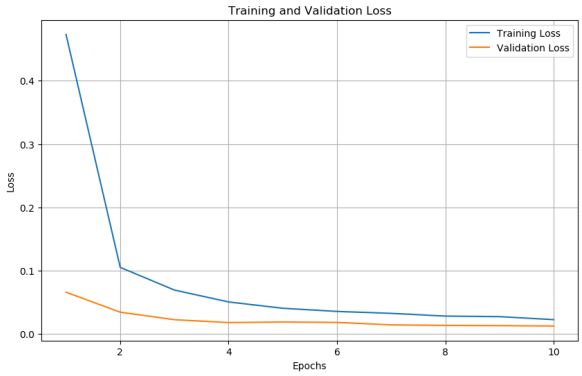

Figure 6: Loss vs Epochs

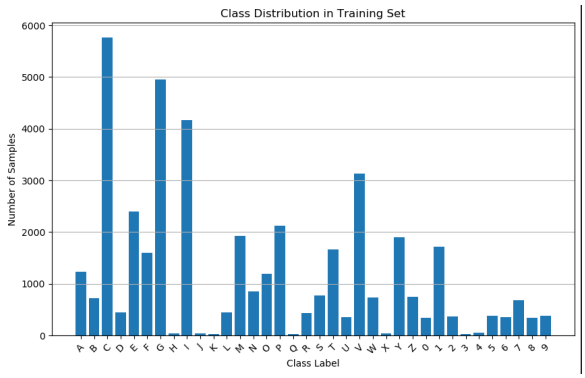

Figure 7: Class Distribution

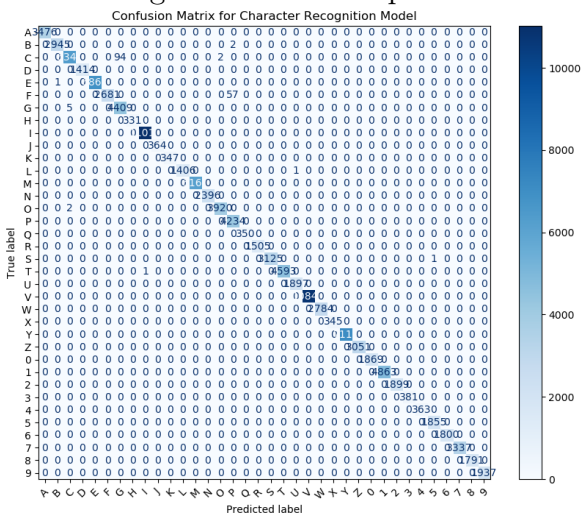

Figure 8: Confusion Matrix

6.5 Error Analysis

Misclassification analysis revealed infrequent errors between visually similar characters (e.g., `O` and `0`, `I` and `1`). Future improvements could include data augmentation or collecting more samples for underrepresented classes.

For tunnel navigation, the system calculates alignment errors and coverage percentages to maintain the robot's position. If misaligned, the robot rotates until alignment is achieved. This method ensures smooth transitions between zones and reliable performance in challenging scenarios.

7 Discussion

The project tested different methods to improve the robot's performance. While simpler detection techniques worked well for basic tasks, more advanced approaches, like machine learning algorithms, would make the system even more accurate in the future. However, given the time constraints and lack of knowledge, PID seemed like a better idea.

To improve upon our work our team would look forward to integrating advanced object detection, fine-tuning path planning for smoother movement, and upgrading the GUI to offer better feedback. These improvements would make the system more reliable and adaptable for future challenges.

8 Conclusion

The project successfully met its goals for the *Fizz Detective* competition, allowing the robot to navigate the environment, detect clues, and generally avoid collisions. By using computer vision, proportional control, and machine learning, we built a reliable and flexible system.

The modular design, made the system easy to manage and expand, while the custom GUI provided quick feedback and debugging. Using two cameras—one for driving and one for high-quality clue detection—helped balance speed and accuracy without overloading the system.

We faced challenges like changing lighting, and processing delays, but we found practical solutions. Future improvements could include better object detection, a driving system that uses imitation learning, and more training data for the neural network. Overall, the project was a great learning experience that allowed us to take our coding and logic abilities to the next level.

8.1 Competition Results Review

It was exciting to see everyone's robots in action and the variety of approaches taken to overcome the challenges of the simulation. Unfortunately, our team ran into some bad luck during our first attempt when a collision with the pedestrian occurred—something that had only happened to us a few times before. On our second attempt, however, we successfully navigated the entire course and achieved a perfect score.

References

1 Quigley, M., Gerkey, B., Smart, W. D. (2015). *Programming Robots with ROS: A Practical Introduction to the Robot Operating System*. O'Reilly Media.

2 Bradski, G., Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.

3 OpenCV Documentation. *Open Source Computer Vision Library*. Available at: `https://docs.opencv.org/`

4 ROS Wiki. *Robot Operating System (ROS) Documentation*. Available at: `http://wiki.ros.org/`

5 Gonzalez, R. C., Woods, R. E. (2017). *Digital Image Processing*. Pearson.

6 OpenAI. Discussions and collaborative resources from lab sessions and class materials.

7 Official Gazebo Tutorials. *Simulation of Robots in Gazebo*. Available at: `http://gazebosim.org/tutorials`